4. Java虚拟机篇

内存泄漏和内存溢出

- 内存泄漏
  - 定义
    - 程序在运行过程中，由于某些原因**未能正确释放不再使用的内存**，导致**内存占用持续增加**，最终可能耗尽可用内存。
  - 原因
    - 静态集合
      - 使用静态static数据结构存储对象，且未清理
    - 事件监听
      - 未取消对事件源的监听，导致对象持续被引用
    - 线程
      - 未停止的线程可能持有对象引用，无法被回收
  - 例子
    - 静态属性
      - **如果集合或大量的对象定义为static的，它们会停留在整个应用程序的生命周期当中**
      - 优化
        - 避免使用静态变量存放大量对象
        - 手动清理集合
        - 如果**使用单例，尽量采用懒加载**
    - 未关闭的资源
      - 忘记关闭数据库连接、输入流和session对象等
      - 优化
        - 在finally中进行资源关闭
        - 关闭连接的自身代码不能发生异常
        - Java7以上 可使用try-with-resources进行资源关闭
    - 使用ThreadLocal
      - 内存泄漏
        - 如果线程一直不结束（例如线程池中的线程），这些 **key 为 null 的 Entry** 会一直存在，导致 **value 无法被回收**，从而造成内存泄漏。
        - 回收ThreadLocal实例后，ThreadLocalMap 中的 key 会变为 null，但对应的 value 仍然存在。
      - 优化
        - 使用ThreadLocal提供的**remove方法**，可对当前线程中的value值进行移除
        - 不要使用ThreadLocal.set(null) 的方式清除value，它实际上并没有清除值，而是查找与当前线程关联的Map并将键值对分别设置为当前线程和null
        - 最好将ThreadLocal视为需要在**finally块**中关闭的资源，以确保即使在发生异常的情况下也始终关闭该资源
- 内存溢出
  - 定义
    - 程序在申请内存时，**没有足够的内存可供分配**，导致程序抛出 OutOfMemoryError 异常。
  - 原因
    - 大量对象创建
      - 程序中不断创建大量对象，超出JVM堆的限制
    - 持久引用
      - 大型数据结构长时间持有对象引用，导致内存累积
    - 递归调用
      - 深度递归导致栈溢出
  - 例子
    - 堆内存溢出
      - 解决方案
        - 增大堆内存（不推荐）
        - 限制对象增长，避免无限制创建对象
    - 方法区（元空间）溢出
      - 解决方案
        - 增加元空间大小（临时方案）
        - 避免动态生成过多类，使用**缓存已生成的类**以减少类加载
    - 直接内存溢出
      - 解决方案
        - 限制直接内存（临时方案）
        - 手动释放内存

类初始化和类加载

- 创建对象的过程
- 类加载检查
  - 如果没有，那必须先执行相应的类加载过程
  - 虚拟机遇到一条 new 指令时，首先将去检查这个指令的参数是否能在**常量池**中定位到一个**类的符号引用**，并且检查这个符号引用代表的类**是否已被加载过、解析和初始化过。**
- 分配内存
  - 虚拟机将为新生对象分配内存
- 初始化零值
  - 虚拟机需要将分配到的内存空间都初始化为零值（不包括对象头）
  - 这一步操作保证了对象的实例字段在 Java 代码中**可以不赋初始值就直接使用**，程序能访问到这些字段的数据类型所对应的零值。
- 进行必要设置，比如对象头
  - 例如这个**对象是哪个类的实例**、如何才能找到类**的元数据信息**、**对象的哈希码**、**对象的 GC 分代年龄等信息。**
  - 这些信息存放在对象头中。另外，根据虚拟机当前运行状态的不同，如是否启用偏向锁等，对象头会有不同的设置方式。
- 执行 init 方法
  - **构造函数**，即class文件中的方法还没有执行，所有的字段都还为零
  - 对象需要的其他资源和状态信息还没有按照预定的意图构造好
- 种类
  - 启动类加载器
    - 作用
      - 负责加载 JVM **核心类库**，这些类通常位于 JAVA_HOME/lib 目录下
      - 如 java.lang.*、java.util.* 等
    - 特点
      - 用 C/C++ 实现，是 JVM 的一部分，不是 Java 类
      - 是类加载器层次结构的顶层，没有父类加载器
  - 扩展类加载器
    - 作用
      - 负责加载扩展类库，这些类通常位于 JAVA_HOME/lib/ext 目录下。
      - 如 javax.* 等
    - 特点
      - 是 Java 类，继承自 java.lang.ClassLoader
      - 父类加载器是启动类加载器。
  - 系统类加载器 / 应用程序类加载器
    - 作用
      - 负责加载**应用程序类路径**（ClassPath）下的类，即**用户编写**的类。
    - 特点
      - 继承自 java.lang.ClassLoader
      - 父类加载器是扩展类加载器。
  - 自定义类加载器
    - 作用
      - 用户可以根据需要自定义类加载器，实现特定的类加载逻辑（如从网络、数据库、加密文件等加载类）
    - 特点
      - 继承自 java.lang.ClassLoader，通常重写 findClass() 方法
      - 父类加载器是应用程序类加载器
- 双亲委派模型
  - 工作原理
    - 当一个类加载器收到类加载请求时，**首先将请求委派给父类加载器**
    - 如果父类加载器**无法加载**，**子类**加载器**才会尝试加载**
  - 作用
    - 保证核心类库安全
      - 避免开发者自定义类**覆盖核心类库**的可能
    - 避免类的重复加载
      - 父类加载器加载的类，子类加载器不会重复加载
    - 提高类加载的效率
      - 如果父类加载器已经加载过了，就直接使用，避免重复查找和解析
  - 如何跳开
    - 通过 自定义类加载器 来跳开它
      - 适用于插件系统、热更新等场景
      - findClass(name) 方法 直接加载类，不交给父类加载器，从而绕开了双亲委派机制
      - 重写 findClass() 方法，不调用 super.loadClass()
    - 重写 loadClass() 方法，修改加载逻辑
      - 特定包名的类 由 findClass() 直接加载
    - 插件机制
      - 加载外部 JAR，避免和应用 JAR 冲突
    - 热加载
      - 允许代码修改后不重启应用
      - Tomcat、Spring Boot
      - Tomcat 自定义 WebAppClassLoader 以隔离 WEB-INF/lib
- 类加载过程
  - 加载
    - 加载阶段的任务是通过类的全限定名（包名 + 类名）来**获取类的字节码**文件，并将其**加载到 JVM 中。**
  - 验证
    - 验证阶段的任务是确保加载的字节码文件是符合**JVM规范**的，不会对 JVM 的安全和稳定性造成威胁
    - 文件格式校验
    - 元数据验证
    - 字节码验证
    - 符号引用验证
  - 连接
    - 准备
      - 准备阶段的任务是为类的**静态变量分配内存**，并**设置默认初始值**
      - 比如int类型初始值是0
      - 被final修饰的static字段不会设置，因为final在编译的时候就分配了
    - 解析
      - 解析阶段是虚拟机将**常量池**的**符号引用**直接替换为**直接引用**的过程
      - 符号引用
        - 符号引用是一种**描述性的引用**，用于在**编译时表示**类、方法、字段等目标。它**不直接指向**目标的**内存地址**，而是通过**符号名称**（如全限定名）来**描述目标。**
        - java.lang.String str = new java.lang.String("Hello");
        - java.lang.String 是一个符号引用，描述了目标类
      - 直接引用
        - 直接引用是运行时的引用，**直接指向目标的内存地址**（如方法区中的类、堆中的对象实例等）
  - 初始化
    - 初始化阶段的任务是执行类的**静态初始化块和静态变量的赋值**操作
  - 使用
    - 在类加载完成后，JVM可以使用该类来创建对象、调用方法、访问字段等
  - 卸载
    - 定义
      - 卸载阶段的任务是将**不再使用**的类从JVM中移除，释放内存资源
    - 可以被卸载的条件包括
      - 类的**所有实例**都已经被回收
      - 加载该类的**类加载器**已经被回收
      - 该类对应的**java.lang.Class对象**没有被任何地方引用
- 类加载器
  - 启动类加载器（Bootstrap ClassLoader）
  - 扩展类加载器（Extension ClassLoader）
  - 应用程序类加载器（Application ClassLoader）
  - 自定义类加载器（User ClassLoader）

垃圾回收

- 基本概念
  - 垃圾回收（Garbage Collection, GC）是**自动管理内存的一种机制**，它负责**自动释放不再被程序引用的对象所占用的内存**
- 触发条件
  - 内存不足时
    - 当JVM检测到堆内存不足，无法为新的对象分配内存时，会自动触发垃圾回收。
  - 手动请求
    - 虽然垃圾回收是自动的，开发者可以通过调用 System.gc() 或 Runtime.getRuntime().gc() 建议 JVM 进行垃圾回收，并不能保证立即执行。
  - JVM参数
    - 启动 Java 应用时可以通过 JVM 参数来调整垃圾回收的行为
    - 比如：-Xmx（最大堆大小）、-Xms（初始堆大小）等
  - 对象数量或内存使用达到阈值
    - 垃圾收集器内部实现了一些策略，以监控对象的创建和内存使用，达到某个阈值时触发垃圾回收。
- 判断垃圾的方法
  - 引用计数法
    - 原理
      - 为每个对象分配一个引用计数器，每当有一个地方引用它时，计数器加1；当引用失效时，计数器减1。
      - 当计数器为0时，表示对象不再被任何变量引用，可以被回收。
    - 缺点
      - **循环引用**
      - 即**两个对象相互引用**，但不再被其他任何对象引用，这时引用计数器不会为0，导致对象无法被回收。
  - 可达性分析算法
    - 原理
      - 从一组称为GC Roots（垃圾收集根）的对象出发，向下追溯它们引用的对象，以及这些对象引用的其他对象，以此类推
      - 如果一个对象到GC Roots没有任何引用链相连（即从GC Roots到这个对象不可达），那么这个对象就被认为是不可达的，可以被回收
    - GC Roots对象包括
      - 虚拟机栈（栈帧中的本地变量表）中引用的对象
      - 方法区中类静态属性引用的对象
      - 本地方法栈中JNI（Java Native Interface）引用的对象
      - 活跃线程的引用
- 垃圾回收算法
  - 定义
    - 是用于**自动管理内存的算法**
    - 自动检测和回收不再使用的对象，从而释放它们所占用的内存空间
  - 解决什么问题
    - 避免内存泄漏
      - 一些对象被分配了内存却无法被释放，导致内存资源的浪费
    - 防止内存溢出
      - 程序需要的内存超过了可用内存的情况
  - 常见的垃圾回收算法
    - 标记-清除算法
      - 过程
        - 标记
          - 从根对象开始，遍历所有可达对象，并标记它们为“存活”
        - 清除
          - 遍历整个堆内存，回收所有未被标记的对象
      - 缺点
        - 效率问题
          - 标记和清除的过程效率都不高
        - 内存碎片
          - 标记清除之后会产生大量不连续的内存碎片，空间碎片太多可能会导致在申请大块内存的时候因为**没有足够的连续空间**导致再次 GC
    - 复制算法
      - 过程
        - 将内存分为两个区域：From区和To区
        - 标记
          - 从根对象开始，标记所有存活对象。
        - 复制
          - 将所有存活对象从From区复制到To区
        - 清除
          - 清空From区，交换From区和To区的角色
      - 优点
        - 解决了内存碎片问题
      - 缺点
        - 每次在申请内存时，都只能使用一半的内存空间，**内存利用率严重不足。**
    - 标记-整理算法
      - 过程
        - 标记
          - 与标记-清除算法相同，标记所有存活对象
        - 整理
          - 将所有存活对象移动到内存的一端，整理内存空间
        - 清除
          - 回收剩余的内存空间
      - 优点
        - 解决了内存碎片问题
      - 缺点
        - 整理过程**需要移动对象**，增加了开销
    - 分代回收算法
      - 思想
        - 根据对象的生命周期将内存分为不同的区域（如年轻代、老年代）。
        - 对不同区域采用不同的垃圾回收算法
      - 年轻代
        - 使用复制算法，因为年轻代中的对象生命周期较短，大部分对象会很快被回收。
      - 过程
        - 老年代
          - 使用标记-清除或标记-整理算法，因为老年代中的对象生命周期较长。
- 垃圾回收器
  - 垃圾回收算法哪些阶段会stop the world
    - STW是指 JVM **暂停所有应用线程**，以便垃圾回收器能够安全地执行内存清理和整理工作
    - 标记阶段停顿分析
      - 初始标记阶段
        - 初始标记阶段是指从GC Roots出发标记全部直接子节点的过程，该阶段是STW的
        - 由于GC Roots数量不多，通常该阶段耗时非常短
        - STW 的原因
        - 需要确保 GC Roots 的引用关系在标记时是稳定
      - 并发标记阶段
        - 并发标记阶段是指从GC Roots开始对堆中对象进行可达性分析，找出存活对象，该阶段是并发的，即应用线程和GC线程可以同时活动。并发标记耗时相对长很多，但因为不是STW，所以我们不太关心该阶段耗时的长短。
      - 再标记阶段
        - 重新标记那些在并发标记阶段发生变化的对象。该阶段是STW的
        - 需要重新扫描并标记在并发标记阶段被修改的对象。
        - STW 的原因
    - 清理阶段停顿分析
      - 清理阶段清点出有存活对象的分区和没有存活对象的分区，该阶段不会清理垃圾对象，也不会执行存活对象的复制。该阶段是STW的。
      - STW 的原因
      - 需要确保在清理过程中，应用线程不会修改对象的引用关系。
    - 复制阶段停顿分析
      - 复制算法中的转移阶段需要分配新内存和复制对象的成员变量。转移阶段是STW的
      - 其中内存分配通常耗时非常短，但对象成员变量的复制耗时有可能较长，这是因为复制耗时与存活对象数量与对象复杂度成正比。对象越复杂，复制耗时越长。
      - STW 的原因
      - 需要确保在整理过程中，应用线程不会修改对象的引用关系。
    - G1的混合回收过程可以分为标记阶段、清理阶段和复制阶段。
  - 垃圾回收器
    - 新生代单线程收集器，标记和清理都是单线程，优点是简单高效
      - Serial收集器（复制算法）
    - 老年代单线程收集器，Serial收集器的老年代版本
      - Serial Old收集器 (标记-整理算法)
    - 新生代收并行集器，实际上是Serial收集器的多线程版本，在多核CPU环境下有着比Serial更好的表现
      - ParNew收集器 (复制算法)
    - 新生代并行收集器，追求高吞吐量，高效利用 CPU。吞吐量 = 用户线程时间/(用户线程时间+GC线程时间)，高吞吐量可以高效率的利用CPU时间，尽快完成程序的运算任务，适合后台应用等对交互相应要求不高的场景
      - Parallel Scavenge收集器 (复制算法)
    - 老年代并行收集器，吞吐量优先，Parallel Scavenge收集器的老年代版本
      - Parallel Old收集器 (标记-整理算法)
    - 老年代并行收集器，以获取**最短回收停顿时间为目标**的收集器，具有高并发、低停顿的特点，追求最短GC回收停顿时间
      - CMS(Concurrent Mark Sweep)收集器 (标记-清除算法)
    - Java堆并行收集器，G1收集器是JDK17提供的一个新收集器，G1收集器基于“标记-整理”算法实现，不会产生内存碎片，G1回收的范围是整个Java堆(包括新生代，老年代)，而前六种收集器回收的范围仅限于新生代或者老年代
      - G1(Garbage First)收集器 (标记-整理算法)
      - 特点
        - G1最大的特点是引入分区的思路，弱化了分代的概念
        - 合理利用垃圾收集各个周期的资源，解决了其他收集器，甚至 CMS 的众多缺陷
        - 停顿时间可控
          - G1可以通过设置预期停顿时间来控制垃圾收集时间避免应用雪崩现象
        - 并行与并发
          - G1 能更充分的利用 CPU 多核环境下的硬件优势，来缩短stw的停顿时间
    - 垃圾回收器
      - CMS 和 G1 的区别
        - 使用的范围不一样
          - CMS收集器是**老年代**的收集器，可以配合新生代的Serial和ParNew收集器一起使用
          - G1收集器收集范围是**老年代和新生代**。不需要结合其他收集器使用
        - STW的时间
          - CMS收集器**以最小的停顿时间**为目标的收集器
          - G1收集器可预测垃圾回收 (opens new window)的停顿时间
        - 垃圾碎片
          - CMS收集器是使用“标记-清除”算法进行的垃圾回收，容易产生内存碎片
          - G1收集器使用的是“标记-整理”算法，进行了空间整合，没有内存空间碎片
        - 垃圾回收的过程不一样

CMS收集器	G1收集器
1. 初始标记	1.初始标记
2. 并发标记	2. 并发标记
3. 重新标记	3. 最终标记
4. 并发清楚	4. 筛选回收

      - CMS会产生浮动垃圾
        - CMS产生浮动垃圾过多时会退化为serial old，效率低，因为在上面的第四阶段，CMS清除垃圾时是并发清除的，这个时候，垃圾回收线程和用户线程同时工作会产生浮动垃圾，也就意味着CMS垃圾回收器必须预留一部分内存空间用于存放浮动垃圾
        - 而G1没有浮动垃圾，G1的筛选回收是多个垃圾回收线程并行gc的，没有浮动垃圾的回收，在执行并发清理步骤时，用户线程也会同时产生一部分可回收对象，但是这部分可回收对象只能在下次执行清理是才会被回收，如果在清理过程中预留给用户线程的内存不足就会出现‘Concurrent Mode Failure’,一旦出现此错误时便会切换到SerialOld收集方式。
      - 什么情况下使用CMS，什么情况使用G1
        - CMS适用场景
          - 适用于对停顿时间要求敏感的应用程序
            - 低延迟需求
          - 主要针对老年代的垃圾回收
            - 老生代收集
          - 容易出现内存碎片，可能需要定期进行Full GC来压缩内存空间
            - 碎片化管理
        - G1适用场景
          - 适用于需要管理大内存堆的场景，能够有效处理数GB以上的堆内存
            - 大堆内存
          - G1通过紧凑整理来减少内存碎片，降低了碎片化对性能的影响
            - 对内存碎片敏感
          - G1在提供较低停顿时间的同时，也保持了相对较高的吞吐量
            - 比较平衡的性能
      - 扩展问题
        - 堆
          - 堆是用于存储对象实例的内存区域。大部分的垃圾回收工作都发生在堆上，因为大多数对象都会被分配在堆上，而垃圾回收的重点通常也是回收堆中不再被引用的对象，以释放内存空间
        - 方法区
          - 方法区是用于存储类信息、常量、静态变量等数据的区域。虽然方法区中的垃圾回收与堆有所不同，但是同样存在对不再需要的常量、无用的类信息等进行清理的过程
        - 作用范围
          - 垃圾回收机制
            - Minor GC (Young GC)
              - 作用范围
                - 只针对年轻代进行回收，包括Eden区和两个Survivor区
              - 触发条件
                - 当Eden区空间不足时，JVM会触发一次Minor GC，将Eden区和一个Survivor区中的存活对象移动到另一个Survivor区或老年代
              - 特点
                - 通常发生得非常频繁，因为年轻代中对象的生命周期较短，回收效率高，暂停时间相对较短
            - Major GC (Old GC)
              - 作用范围
                - 主要针对老年代进行回收，但不一定只回收老年代
              - 触发条件
                - 当老年代空间不足时，或者系统检测到年轻代对象晋升到老年代的速度过快，可能会触发Major GC
              - 特点
                - 相比Minor GC，Major GC发生的频率较低，但每次回收可能需要更长的时间，因为老年代中的对象存活率较高
            - Full GC
              - 作用范围
                - 对整个堆内存 (包括年轻代、老年代)以及永久代(元空间) 进行回收
              - 触发条件
                - 显式调用 System.gc()
                  - 虽然不能保证立即执行，但JVM会尝试执行Full GC
                - **老年代空间不足**
                  - 当老年代无法容纳从新生代晋升的对象时
                - 方法区空间不足
                  - 当方法区空间不足时
              - 特点
                - Full GC是最昂贵的操作，因为它需要停止所有的工作线程（STW），遍历整个堆内存来查找和回收不再使用的对象，因此应尽量减少Full GC的触发
        - 什么时候元空间的内容会被清理
          - 类加载器被回收
            - JVM 只有在发现某个类加载器不可达时，才会卸载该加载器加载的所有类，并释放相关的 元空间
          - 手动触发 GC
            - 通过 System.gc() 或 Full GC，可能会触发 类卸载（Class Unloading），但不会立即清理元空间
        - 扩展问题

内存模型

直接内存

- 属于**操作系统的本地内存**，是可以直接操作的
- 是一种在 Java 中使用 堆外内存（Off-Heap Memory）的方式，主要是指不经过 JVM 堆 而直接向 操作系统 申请的内存空间
- 它通常通过 NIO（New Input/Output）提供的 ByteBuffer 类来分配和管理
  - NIO
    - Java 中代表 New Input/Output，是 Java 1.4 引入的一套新的 I/O 操作方式，旨在替代传统的 BIO（Blocking I/O，阻塞 I/O）
  - NIO 支持更高效的 I/O 操作
    - 非阻塞 I/O（Non-blocking I/O）
    - 内存映射文件（Memory-Mapped File）
    - 直接内存（Direct Memory）
    - 多路复用（Selector）
- 特点
  - 不占用堆空间
    - 直接使用操作系统的内存，不受 GC 影响，减少了 GC 压力
  - 读写性能更高
    - 适用于 I/O 场景，数据可以直接与操作系统的通道（Channel）交互，避免了堆内存与内核空间的多次拷贝，提升了性能
  - **需要手动释放**
    - 由于不受 GC 控制，分配的直接内存需要及时手动释放，否则可能引发内存泄漏
- 使用场景
  - NIO 直接缓冲区（DirectByteBuffer）
  - 高性能通信框架
    - Netty、Kafka、RocketMQ 等高性能网络通信框架大量使用 直接内存，减少 GC 影响，提升吞吐量
  - 数据库缓存
    - Ehcache、RocksDB 允许使用 直接内存 作为缓存，提高查询速度
  - JNI
    - Java 调用 C/C++ 代码时，需要分配和管理本地内存
- 堆外内存定义
  - 由操作系统管理，不受 GC 影响，需要手动释放

运行时数据区

- 元空间（方法区）
  - 是 Java 8 及以后版本中引入的一种用于替代 永久代（PermGen）的 方法区 实现。
  - 存储内容
    - 类的元数据
      - 类的基本信息
        - 类名、访问修饰符（如 public、final）
        - 类的父类、实现的接口
        - 类的字段（成员变量）和方法信息（方法名、参数类型、返回类型等）
        - 类的注解信息
      - 类的结构信息
        - 类的字节码（方法代码）
        - 类的属性表（如 SourceFile、InnerClasses 等）
      - 类的加载器引用
        - 加载该类的类加载器的引用，用于支持类的卸载和垃圾回收
        - 每个类加载器都有独立的元空间，类卸载时自动回收元数据
    - 运行时常量池(Java8之后移到了堆)（Runtime Constant Pool）
      - 符号引用
        - 类和接口的全限定名（如 java/lang/String）。
        - 字段的名称和描述符（如 I:java/lang/String:)。
        - 方法的名称和描述符（如 ()V 表示无参数、无返回值的方法）。
      - 字面量
        - 字符串常量（如 “Hello, World!”）。
        - 数字常量（如 10、3.14）——但这里的数字常量是作为符号引用或字面量存储的，而不是基本数据类型的常量。
        - 布尔常量（如 true、false）。
    - 反射和动态代理
      - 存储由反射和动态代理生成的类信息
  - 为什么使用元空间代替永久代？
    - 避免内存不足（OOM）
      - 永久代 受 JVM 固定大小限制，容易出现 java.lang.OutOfMemoryError: PermGen space
      - 元空间 使用堆外内存，理论上只受物理内存限制，能动态扩展，减少 OOM 风险
    - 简化 JVM 内存管理
      - 取消永久代，类元数据 由 元空间 统一管理，简化了 JVM 内存模型，提升了可维护性
- Java 虚拟机栈
  - 由 多个栈帧（Stack Frame）组成，每个方法的调用 都会对应一个栈帧。方法执行结束后栈帧会被销毁，遵循 “先进后出” (FILO) 的原则。
  - 栈帧的组成
    - 局部变量表（Local Variables）
      - 存储方法执行过程中的局部变量（基本数据类型、对象引用、this）
      - Slot：局部变量表中的每个变量占用一个 Slot（32 位）
      - 64 位数据类型（long 和 double）占用 两个 Slot
    - 操作数栈（Operand Stack）
      - 用于执行 字节码指令，在方法执行过程中进行操作数的压入（push）和弹出（pop）
    - 动态链接（Dynamic Linking）
      - 存储方法执行时指向 常量池 的引用，支持 动态绑定 和 指向方法
    - 方法出口（Return Address）
      - 记录方法执行完毕后的返回地址，指向调用该方法的位置
  - 工作原理
    - 方法调用
      - 每调用一个方法，JVM 栈中都会压入一个新的栈帧
    - 方法执行
      - 在栈帧中执行方法，利用局部变量表、操作数栈进行计算
    - 方法返回
      - 方法执行结束后，栈帧出栈，控制权返回到上一个栈帧
- 本地方法栈
  - 用于支持 Native 方法（即使用其他语言如 C、C++ 编写并调用的本地代码），每个线程在创建时，除了 Java 虚拟机栈，还会拥有一个独立的 本地方法栈。
  - 作用
    - 执行 Native 方法
      - 调用使用 JNI（Java Native Interface） 定义的本地方法。
      - 例如，Java 中的 System.gc()、Object.hashCode()、ClassLoader 都依赖于本地方法实现。
    - 与操作系统交互
      - 通过本地方法与底层系统资源（如文件系统、网络、设备驱动）交互。
    - 扩展 JVM 功能
      - 可使用 C/C++ 扩展 Java 功能，如调用系统 API、硬件接口等。
- 程序计数器
  - 每个线程都会有一个**独立的**程序计数器，用于记录当前线程执行的 **字节码指令地址**
  - 在多线程环境中用于 **线程切换** 时**恢复正确的执行位置。**
  - 作用
    - 记录当前执行的字节码指令地址
      - Java 方法
        - 记录执行的 字节码指令地址（JVM 指令）
      - Native 方法
        - **程序计数器值为空**（undefined），因为本地方法不在 JVM 中执行
    - 线程切换和恢复
      - Java 是多线程环境，CPU 在多个线程之间来回切换，程序计数器 确保线程恢复时能回到正确的位置继续执行
    - 控制程序流转
      - 管理 分支、循环、异常处理、方法返回 等流程控制，依赖于程序计数器的更新
  - 为什么程序计数器是私有的？
    - 线程独立性
      - 每个线程都是独立执行的，它们可能执行不同的代码或处于**不同的执行阶段。**
      - 如果程序计数器是共享的，**多个线程会互相干扰**，导致执行位置混乱。
    - 线程切换的需要
      - 在多线程环境下，CPU 需要在线程之间切换执行。
      - 每个线程的程序计数器需要独立保存和恢复，以确保线程切换后能够继续从正确的位置执行。
    - 方法调用的独立性
      - 每个线程的**方法调用栈是独立的**，程序计数器需要记录每个线程的**调用位置。**
      - 如果程序计数器是共享的，**不同线程的方法调用会互相干扰**，导致返回位置错误。
- 堆内存
  - 主要用于存放对象实例和数组，也是 Java 垃圾回收（GC）的重点管理区域，几乎所有的 Java 对象都在堆中分配。生命周期由 GC 自动管理。
    - 作用
      - 存储对象实例、数组，管理对象生命周期
    - 线程共享
      - 堆内存是所有线程共享的内存区域
    - 生命周期
      - 随 JVM 启动而创建，随 JVM 退出而销毁
    - GC 管理
      - 由 垃圾回收器（GC） 自动管理和回收
    - 异常
      - 当堆空间不足时，会抛出 OutOfMemoryError
    - 分配方式
      - new 关键字、反射、序列化、数组
  - JDK 1.8 后，**字符串常量池**和**运行时常量池**从永久代中剥离出来，存放在堆中。
  - 构成

    - 新生代（Young Generation）
      - Eden Space
        - 大多数新创建的对象首先存放在这里
        - Eden区相对较小，当Eden区满时，会触发一次Minor GC（新生代垃圾回收）
      - Survivor Space
        - 在每次Minor GC后，存活下来的对象会被移动到其中一个Survivor空间，以继续它们的生命周期。
        - 这两个区域轮流充当对象的中转站，帮助区分短暂存活的对象和长期存活的对象。
        - S0
        - S1
    - 老年代（Old Generation/Tenured Generation）
      - 经过多次 GC 仍存活的对象（年龄达到阈值 默认 15）会被移动到老年代
        - 老年代中的对象生命周期较长，因此Major GC（也称为Full GC）发生的频率相对较低，但其执行时间通常比Minor GC长。
        - 老年代的空间通常比新生代大，以存储更多的长期存活对象
      - 大对象区（Large Object Space / Humongous Objects）
        - 在某些JVM实现中（如G1垃圾收集器），为大对象分配了专门的区域，称为大对象区或Humongous Objects区域。
        - 大对象是指需要**大量连续内存空间**的对象，如大数组
        - 这类对象直接分配在老年代，以避免因频繁的年轻代晋升而导致的内存碎片化问题。
- 堆和栈区别
  - 用途
    - 栈主要用于存储局部变量、方法调用的参数、方法返回地址以及一些临时数据。
    - 每当一个方法被调用，一个栈帧就会在栈中创建，用于**存储该方法的信息**。当方法**执行完毕**，栈帧也会**被移除**
    - 当我们在栈中讨论“存储”时，实际上指的是存储基本类型的数据和对象的引用，而不是对象本身。
  - 栈 Stack
    - 生命周期
      - 栈中的**数据**具有确定的生命周期
      - 当一个方法调用结束时，其对应的栈帧就会被销毁，栈中存储的局部变量也会随之消失
    - 存取速度
      - 栈的存取速度通常比堆快，因为栈遵循先进后出的原则，操作简单快速
    - 存储空间
      - 栈的空间相对较小，且**固定**，由**操作系统管理**
      - 当栈溢出时，通常是因为递归过深或局部变量过大
    - 可见性
      - 栈中的数据对线程是**私有的**，每个线程有自己的栈空间
  - 堆 Heap
    - 用途
      - 堆用于存储对象的实例（包括类的实例和数组）
      - 当你使用new关键字创建一个对象时，对象的实例就会在堆上分配空间。
    - 生命周期
      - 堆中的**对象**生命周期不确定
      - 对象会在垃圾回收机制检测到对象不再被引用时才被回收
    - 存取速度
      - 堆的存取速度相对较慢，因为对象在堆上的**分配和回收需要更多的时间**，**垃圾回收机制的运行**也会影响性能。
    - 存储空间
      - 堆的空间较大，**动态扩展**，**由JVM管理**
      - 堆溢出通常是由于创建了太多的大对象或未能及时回收不再使用的对象
    - 可见性
      - 堆中的数据对线程是**共享的**，所有线程都可以访问堆上的对象
- 方法区中方法的执行过程
  - 主要涉及 方法区、栈 和 程序计数器 的协作
  - 解析方法调用
    - JVM会根据方法的**符号引用**找到**实际的方法地址**（如果之前没有解析过的话）
  - 栈帧创建
    - 在调用一个方法前，JVM会在当前线程的Java虚拟机栈中为**该方法分配一个新的栈帧**，用于存储局部变量表、操作数栈、动态链接、方法出口等信息。
  - 执行方法
    - 执行方法内的**字节码指令**，涉及的操作可能包括局部变量的读写、操作数栈的操作、跳转控制、对象创建、方法调用等。
  - 返回处理
    - 方法执行完毕后，可能会返回一个结果给调用者，并清理当前栈帧，恢复调用者的执行环境。
- String(保存在哪里
  - 字符串常量池
    - 在 JDK 1.7 及之前，字符串常量池是方法区（永久代）的一部分
    - 在 JDK 1.8 及之后，字符串常量池被移到 堆 中
- String s = new String（“abc”）执行过程中分别对应哪些内存区域？
  - 检查字符串常量池 | 永久代 / 堆
    - JVM 首先检查字符串常量池中是否存在字面量 “abc”
      - 如果存在，直接引用常量池中的 “abc”。
      - 如果不存在，将 “abc” 添加到字符串常量池中。
  - 创建新的 String 对象 | 堆
    - 使用 new 关键字创建一个新的 String 对象。
    - 这个对象存储在堆中，其内部的 char[] 数组引用字符串常量池中的 “abc”。
  - 分配局部变量 | 栈
    - 将新创建的 String 对象的引用赋值给局部变量 s。
    - s 是一个局部变量，存储在栈中。
- 内存溢出
  - 堆内存溢出
    - 堆内存不足以容纳新创建的对象
    - Java.lang.OutOfMemoryError:Java heap space异常
  - 栈溢出
    - 栈内存不足以容纳**方法调用的上下文信息**
    - 如果我们写一段程序**不断的进行递归调用**，而且**没有退出条件**，就会导致不断地进行压栈。
    - 类似这种情况，JVM 实际会抛出 StackOverFlowError；
    - 如果 JVM 试图去扩展栈空间的时候失败，则会抛出 OutOfMemoryError。
  - 元空间溢出
    - 元空间（方法区）内存不足以容纳加载的类信息
      - Java.lang.OutOfMemoryError: Metaspace
      - 系统的**代码非常多**或**引用的第三方包**非常多或者通过**动态代码生成类加载**等方法，导致元空间的内存占用很大。
  - 直接内存内存溢出
    - 直接内存不足以容纳 NIO 缓冲区
      - java.lang.outofmemoryerror: Direct buffer memory异常
      - 在使用ByteBuffer中的allocateDirect()的时候会用到，很多JavaNIO(像netty)的框架中被封装为其他的方法

对象的生命周期

- 创建
  - 对象通过关键字new在堆内存中被实例化，构造函数被调用，对象的内存空间被分配
- 使用
  - 对象被引用并执行相应的操作，可以通过引用访问对象的属性和方法，在程序运行过程中被不断使用
- 销毁
  - 当对象不再被引用时，通过垃圾回收机制自动回收对象所占用的内存空间
  - 垃圾回收器会在适当的时候检测并回收不再被引用的对象，释放对象占用的内存空间，完成对象的销毁过程

引用

- 强引用
  - 默认引用类型，对象**不会被垃圾回收。**
  - 它的生命周期由引用变量控制。只要一个对象有强引用指向它，垃圾回收器就不会回收这个对象
  - MyObject obj = new MyObject();
  - 在这个例子中，obj 是对 MyObject 的强引用。只要 obj 仍然存在，MyObject 对象就不会被垃圾回收。
- 软引用
  - 适合缓存，**内存不足时回收。**
    - 软引用通过 SoftReference 类实现
    - 内存不足时，回收软引用指向的对象
- 弱引用
  - 适合弱关联，垃圾回收时**立即回收。**
    - 弱引用通过 WeakReference 类实现
    - 强度比软引用更低一点，弱引用的对象**下一次GC**的时候一定会被回收，而**不管内存是否足够**
      - WeakReference<MyObject> weakRef = new WeakReference<>(new MyObject());
      - 在这个例子中，weakRef 是 MyObject 的弱引用。如果没有其他强引用指向 MyObject 对象，垃圾回收器会在适当的时机回收 MyObject 对象，甚至即使 weakRef 仍然存在。
    - 主要用途
      - **创建非强制性的对象引用**，这些引用可以在内存压力大时被垃圾回收器清理，从而避免内存泄露
    - 使用场景
      - 缓存系统
        - 弱引用常用于实现缓存，特别是当希望缓存项能够在内存压力下自动释放时
        - 如果缓存的大小不受控制，可能会导致内存溢出。使用弱引用来维护缓存，可以让JVM在需要更多内存时自动清理这些缓存对象。
      - 对象池
        - 在对象池中，弱引用可以用来管理那些暂时不使用的对象。当对象不再被强引用时，它们可以被垃圾回收，释放内存。
      - 避免内存泄露
        - 当一个对象不应该被长期引用时，使用弱引用可以防止该对象被意外地保留，从而避免潜在的内存泄露。
    - 注意
      - 一旦对象被垃圾回收，通过弱引用获取的对象可能会变为null，因此在使用前通常需要检查这一点。
- 虚引用 / 幻影引用
  - 用于跟踪对象回收状态，无法获取对象
    - 虚引用通过 PhantomReference 类实现。
    - 虚引用无法通过 get() 方法获取对象，仅用于跟踪对象的垃圾回收状态。
    - 是最弱的引用关系，可以用PhantomReference来描述，他必须和ReferenceQueue一起使用，同样的当发生GC的时候，虚引用也会被回收。可以用虚引用来**管理堆外内存。**

Presented with xmind